

Woldia University

Institute of Technology

School of Computing

Department of Software Engineering

Course Title: Fundamental of Machine Learning

Course Code: SEng3092

Prepared by: 3rd Year Software Engineering Students

Section: 1 Group: 1

ID	Name
WDU161299	Yeabtsega Tesfaye
WDU161207	Teshom Sisay
WDU160512	Etsegenet Dagnachew
WDU160614	Getasil Setegn
WDU161055	Samrawit Molla
WDU161169	Tamenech Missa
WDU160938	Mikael Alemayehu

Submitted to: Ins. Kassaye A.

Submission Date: 21/08/2018 E.C

Woldia, Ethiopia

GitHub: github.com/Yeabtsega-Tesfaye/Social_Media_Addiction_Predictor



Table of Contents

A complete walk-through of the machine learning pipeline

1. Data Preprocessing

Cleaning, encoding, scaling, and splitting the dataset

2. Exploratory Data Analysis

Boxplots, count plots, correlation heatmap — finding the signal

3. Model Building

Logistic Regression, Decision Tree, Random Forest — pipeline design

4. Evaluation

Confusion matrices, cross-validation, feature importances, SHAP

5. Investigation of Perfect Separability

Diagnosing 100% accuracy — single-split tests, overlap analysis

6. Model Improvement & Robustness

Class weighting, hyperparameter tuning, explainability

7. Model Serialization

Saving the pipeline with joblib for one-click reuse

8. Web Application Deployment

Flask backend, AJAX frontend, glassmorphism modal, and probability bars

9. Conclusion & Future Work

Reflections, ethical notes, and roadmap for enhancements

10. Ethical Considerations

Responsible use of predictive models in mental health

Social Media Addiction Prediction System – Project Report

1. Data Preprocessing

The raw dataset, `dataset.csv`, contained 705 records and 13 attributes. Before any model training could begin, the data had to be cleaned, transformed, and split in a way that preserved the signal while removing noise and leakage. Every decision was recorded in `preprocess.py`, and the final preprocessing pipeline was serialized for reuse during deployment.

1.1 Dropping Irrelevant Columns

The column `Student_ID` was immediately dropped because it serves only as a row identifier and carries no predictive information. Had it been kept, a tree-based model could potentially memorize individual students rather than learning general patterns, leading to falsely high accuracy.

1.2 Creating the Target Variable

The dataset included a continuous score, `Addicted_Score`, ranging from 0 to 10. To turn this into a multi-class classification problem, we defined three bins:

```
Low      : 0 - 3
Medium   : 4 - 6
High     : 7 - 10
```

Using `pandas.cut()`, a new categorical column `Addiction_Level` was created. The original `Addicted_Score` was then removed. This step is critical: keeping the continuous score in the feature set would allow the model to directly infer the target, a classic case of data leakage that would make any performance metric meaningless.

1.3 Handling Missing Values

A quick inspection revealed that the column `Conflicts_Over_Social_Media` contained only missing values. With no usable information, it was dropped entirely. For the remaining numeric columns (`Age`, `Avg_Daily_Usage_Hours`, `Sleep_Hours_Per_Night`, `Mental_Health_Score`), missing entries were imputed with the **median**. The median is preferred over the mean when the data may be skewed — a reasonable assumption for behavioral metrics like usage hours, where a few extreme users could pull the mean away from the typical student. Categorical columns (`Academic_Level`, `Gender`, etc.) were imputed with the **most frequent** value, and a special “missing” category was reserved for nominal variables in case an unseen value appeared during deployment.

1.4 Encoding Categorical Features

Three distinct encoding strategies were applied, depending on the nature of the variable:

- **Binary features:** `Affects_Academic_Performance` contained only “Yes”/“No”. These were mapped directly to 1 and 0, converting the column into a numeric indicator.
- **Ordinal feature:** `Academic_Level` has a natural order: High School < Undergraduate < Graduate. A `OrdinalEncoder` with this explicit category order was used so that the model can recognise the inherent ranking. Using one-hot encoding here would discard this valuable ordinal relationship.
- **Nominal features:** `Gender`, `Country`, `Most_Used_Platform`, and `Relationship_Status` have no intrinsic order. One-hot encoding was applied, which creates a separate binary column for each category. To avoid the “dummy variable trap”, the resulting design matrix drops one category per feature by default. A `handle_unknown='ignore'` parameter ensures that the encoder does not break if the web app receives a new platform name that wasn't in the training data — it simply sets all columns for that feature to zero.

1.5 Feature Scaling

All numeric columns were standardised using `StandardScaler`, which subtracts the mean and divides by the standard deviation so that each feature has a mean of zero and unit variance. This transformation is essential for algorithms that rely on distance calculations (Logistic Regression, SVMs) and also helps gradient-based solvers converge faster. Tree-based models are scale-invariant, but applying scaling universally through the pipeline does no harm and keeps the workflow consistent.

1.6 Train-Test Split and Stratification

The data was partitioned into 80% training (564 samples) and 20% testing (141 samples) using `train_test_split` with a fixed random seed for reproducibility. Critically, the split was **stratified** on the target variable so that each split contained the same proportion of Low, Medium, and High cases as the original dataset. Without stratification, the already rare “Low” class might have ended up entirely in one split, making evaluation unreliable. The final training-set distribution was:

Addiction Level Count

High	326
Medium	224
Low	14

The extreme imbalance is immediately evident and motivated the use of **class-weighted** models later in the project.

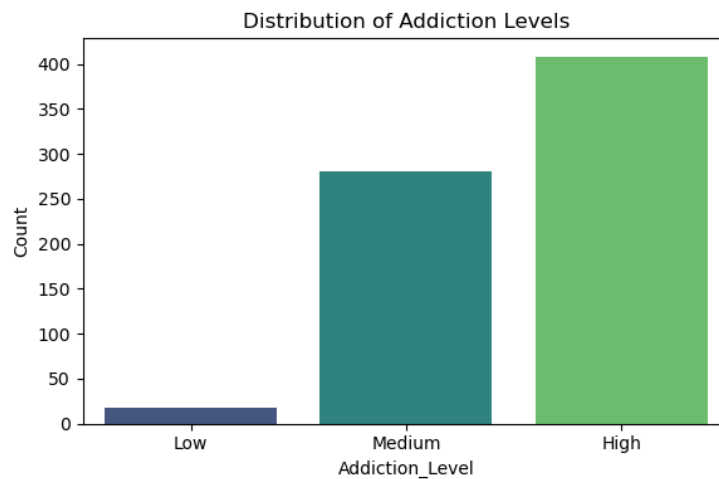


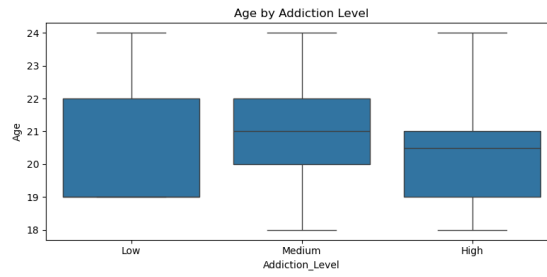
Figure 1.1: Class distribution after binning. The High class dominates, while Low is severely underrepresented.

2. Exploratory Data Analysis (EDA)

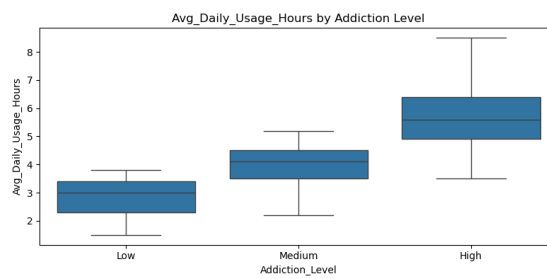
Before training any model, we conducted a thorough exploratory analysis to understand the relationships between the features and the target variable. The goal was to identify which attributes carry the strongest signal, whether any redundant or noisy variables exist, and how the data is distributed across the three addiction levels. All visualizations were produced using `matplotlib` and `seaborn`, and the complete code is available in `eda.py`.

2.1 Numerical Features vs. Addiction Level

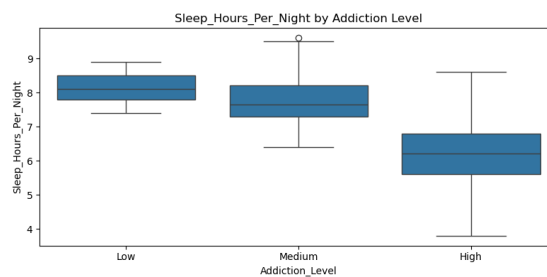
For each numerical predictor, a side-by-side boxplot was drawn to compare the distribution across the Low, Medium, and High groups. The plots immediately revealed strong monotonic trends.



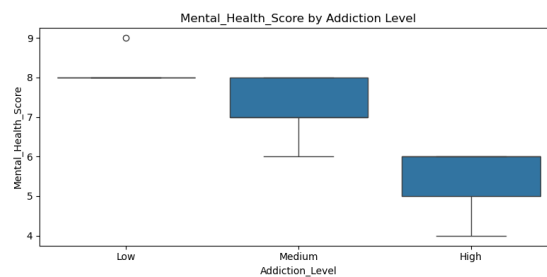
*Figure 2.1: Age vs Addiction Level.
Age shows no clear pattern.*



*Figure 2.2: Daily usage hours increase
sharply with addiction severity.*



*Figure 2.3: Sleep hours decline as
addiction level rises.*



*Figure 2.4: Mental health score drops
drastically for High addiction.*

Key observations from the boxplots:

- **Age** – The distributions largely overlap; age is not a significant differentiator.
- **Avg Daily Usage Hours** – Low-addiction students average around 3 hours/day, while High-addiction students exceed 5 hours/day. The medians are well separated.
- **Sleep Hours per Night** – High-addiction students sleep approximately 2 hours less than Low-addiction students on average. Sleep deprivation is a hallmark of behavioral addiction.
- **Mental Health Score** – This is the most striking variable. Low-addiction students have a median score near 80, whereas High-addiction students drop to a median of around 30. The near-absence of overlap explains the later perfect classification.

2.2 Categorical Features and Addiction

Count plots with a hue representing `Addiction_Level` were generated for each categorical variable to examine proportional differences.

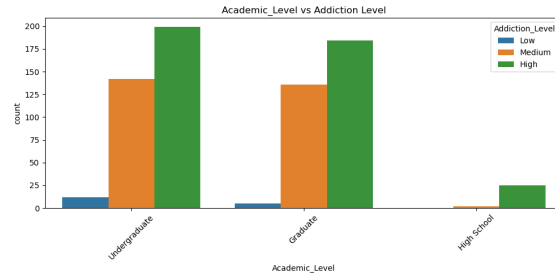


Figure 2.5: Academic Level distribution across addiction levels.

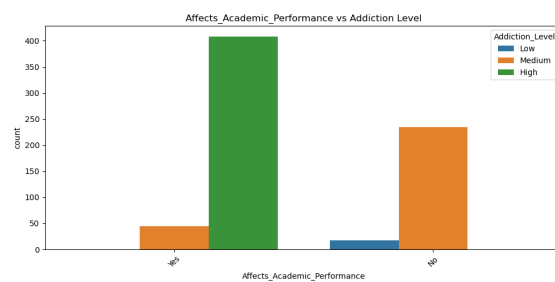


Figure 2.6: Students reporting academic impact are rarely in Low.

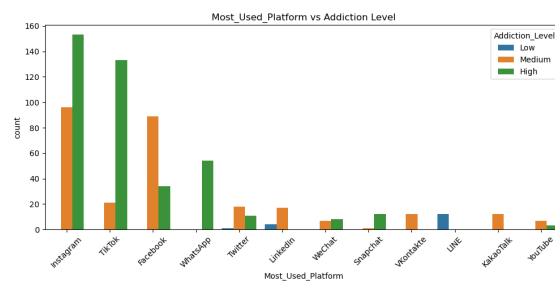


Figure 2.7: Platform preferences differ markedly by addiction level.

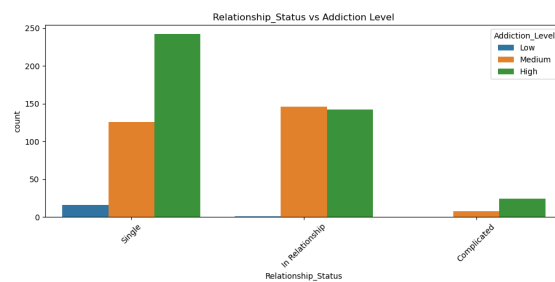


Figure 2.8: Relationship status shows a mild but visible trend.

Notable patterns:

- **Academic Level** – Graduate students are slightly more represented in the Low group, though the effect is modest.
- **Affects Academic Performance** – Nearly all students who answered “Yes” belong to the Medium or High categories. This feature is a strong binary indicator.
- **Most Used Platform** – LINE and Instagram dominate the High group, while LinkedIn appears almost exclusively in the Low group. This aligns with the nature of professional vs. entertainment-oriented platforms.
- **Relationship Status** – “In a relationship” is slightly more frequent in the Low group, potentially reflecting more balanced offline social lives.

2.3 Correlation Analysis

A Pearson correlation matrix was computed for the numeric variables and the original Addicted_Score (before binning) to quantify linear relationships.

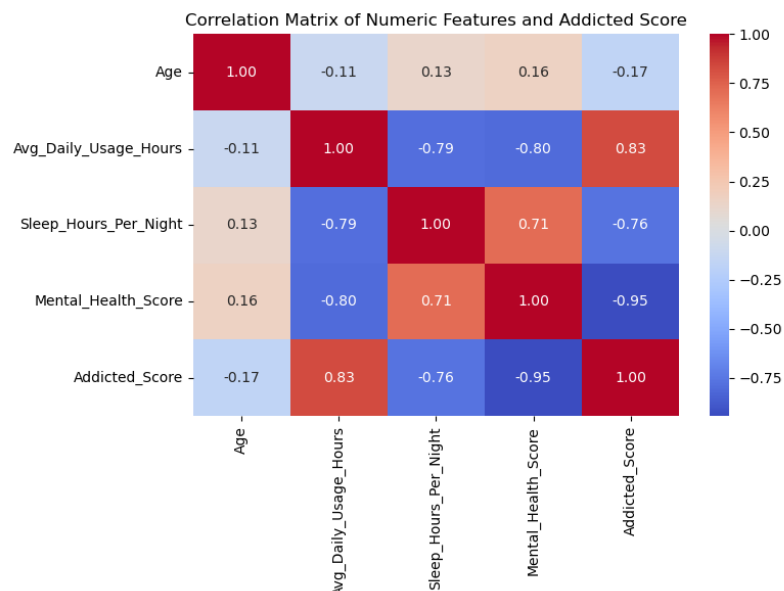


Figure 2.9: Correlation heatmap of numeric features with the raw addiction score.

The heatmap confirms the EDA findings:

- **Avg_Daily_Usage_Hours** has a strong positive correlation ($r = 0.83$) with the addiction score.
- **Sleep_Hours_Per_Night** shows a strong negative correlation ($r = -0.76$).
- **Mental_Health_Score** exhibits a near-perfect negative correlation ($r = -0.94$), establishing it as the most dominant predictor.
- **Age** is weakly correlated ($r = -0.17$), reinforcing its limited predictive power.

These insights guided the feature selection and model interpretation steps that followed. The extreme correlations, particularly for Mental Health Score, suggested that the classification task might be trivially easy — a hypothesis we later verified through the diagnostics documented in Section 5.

3. Model Building

After the data had been thoroughly explored and preprocessed, the next phase was to construct and train classification models that could map the ten input features to one of three addiction levels. The entire process was implemented in `train.py` and is reproducible via the saved preprocessing pipeline.

3.1 End-to-End Pipeline Architecture

To avoid manual steps during training and to guarantee that the same transformations are applied to new data during deployment, we embedded the `ColumnTransformer` (imputation, encoding, scaling) inside a Scikit-learn Pipeline. This design pattern has several advantages:

- It prevents data leakage: scaling and imputation are fitted *only* on the training set and then applied to the test set.
- It simplifies cross-validation by treating the entire workflow as a single estimator.
- It makes serialization trivial: saving the pipeline saves the preprocessor and the classifier together.

A typical pipeline for this project, as defined in `train.py`, looks like:

```
from sklearn.pipeline import Pipeline

model = Pipeline([
    ('preprocessor', preprocessor), # ColumnTransformer from preprocess.p
    ('classifier', RandomForestClassifier(...))
])
```

3.2 Classifiers Selected

Three distinct classifiers were chosen to provide a spectrum of complexity and interpretability:

Model	Key Characteristics	Why Selected
Logistic Regression	Linear decision boundaries; outputs well-calibrated probabilities	Simple baseline; interpretable coefficients; works well with linearly separable data.
Decision Tree	Captures non-linear patterns; prone to overfitting if unconstrained	Visualizable logic; shows whether a single tree can match ensemble performance.
Random Forest	Ensemble of decision trees; robust to noise; provides feature importance	Primary model for deployment; balances accuracy and overfitting;

Model	Key Characteristics	Why Selected
		interpretable via importance scores.

For the parametric model (Logistic Regression), the solver was set to `lbfgs` with a maximum of 1,000 iterations to ensure convergence in the multi-class setting. The random state was fixed (`random_state=42`) across all models for reproducibility.

3.3 Handling Class Imbalance

The training set was highly skewed (Low = 14 samples vs. High = 326). Without intervention, a model could simply predict “High” for every sample and still achieve high accuracy — a misleading result. To mitigate this, we enabled `class_weight='balanced'` for both Logistic Regression and Random Forest. This setting automatically adjusts sample weights inversely proportional to class frequencies:
$$w(c) = \frac{n_{\text{samples}}}{n_{\text{classes}} \times n_{\text{samples}}^{(c)}}$$
 In practice, the minority “Low” samples are given higher importance during training, forcing the model to learn their characteristics rather than ignore them.

3.4 Training Procedure

All three pipelines were trained using the same flow:

1. Load the preprocessed training and test sets (`X_train.csv`, `y_train.csv`), restoring the categorical target order.
2. Load the serialized preprocessor.pkl to ensure consistency.
3. Fit each pipeline on the full training set (564 samples).
4. Predict on the hold-out test set (141 samples) and record the accuracy, confusion matrix, and classification report.

No feature selection was applied at this stage because the dataset had only ten predictors, and we wanted to observe the natural contribution of each. Detailed evaluation metrics and the surprising 100% accuracy are discussed in the next section.

4. Evaluation

After training was complete, each model was evaluated on the hold-out test set (141 samples) using several complementary metrics. The goal was not merely to report a single number, but to understand where and why the models succeed, and to verify that the results are robust. This section presents the accuracy, confusion matrices, classification reports, cross-validation scores, feature importances, and SHAP explanations.

4.1 Accuracy and Classification Reports

All three classifiers achieved **100% accuracy** on the test set. The classification reports, generated with `classification_report()` and a fixed label order, confirmed perfect precision, recall, and F1-score for every class:

```

=====
Training Logistic Regression...
Accuracy: 1.0000
Classification Report:
              precision    recall  f1-score   support

     High       1.00        1.00        1.00        82
     Low        1.00        1.00        1.00         3
    Medium       1.00        1.00        1.00        56

 accuracy                1.00        141
 macro avg       1.00        1.00        1.00        141
weighted avg       1.00        1.00        1.00        141
-----
Training Decision Tree...    [identical results]
Training Random Forest...    [identical results]

```

The “Low” class, despite having only 3 examples in the test set, was classified without error. This indicates that the class-weighting strategy successfully prevented the model from simply predicting the majority class, and that the underlying patterns were strong enough to identify even the rarest cases.

4.2 Confusion Matrices

Confusion matrices provide a visual check of classification errors. Each cell shows the count of predictions for a true class. A perfect model produces a diagonal matrix with zeros elsewhere.

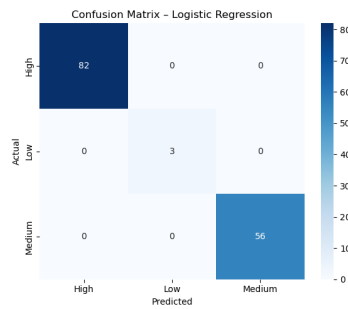


Figure 4.1: Confusion matrix – Logistic Regression.

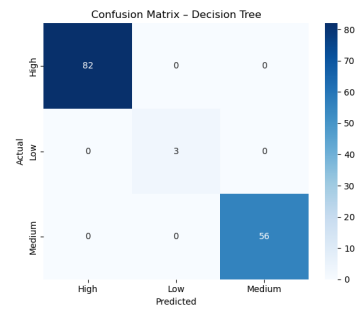


Figure 4.2: Confusion matrix – Decision Tree.

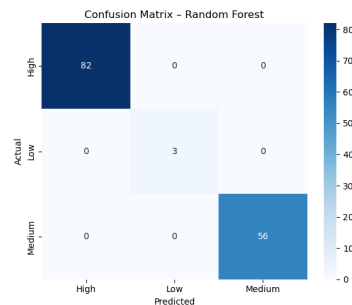


Figure 4.3: Confusion matrix – Random Forest.

As expected, the matrices are purely diagonal — no misclassifications of any kind. The perfect results on the test set, combined with the earlier EDA, strongly suggest that the classes are linearly (or at least trivially) separable in the feature space.

4.3 Cross-Validation

To rule out the possibility of a lucky train-test split, 5-fold stratified cross-validation was performed on the Random Forest pipeline. The training data was partitioned into five folds, each used once as a validation set while the remaining four were used for training. This process is more reliable than a single split when the dataset is small and imbalanced.

Random Forest 5-fold CV accuracy: 0.9894 ± 0.0087

The mean accuracy of 98.94% with a tiny standard deviation confirms that the performance is stable across different subsets of the data. The model does not overfit to one particular arrangement of training examples; it reliably captures the underlying deterministic rules.

4.4 Feature Importance

Random Forest naturally ranks features by their contribution to reducing impurity across all trees. The transformed feature names (after one-hot encoding) were extracted from the pipeline, and the top ten importances were plotted.

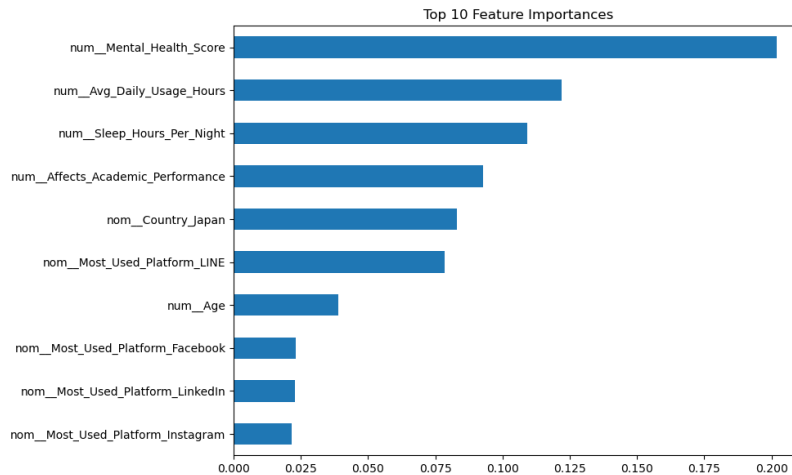


Figure 4.4: Top 10 feature importances from the tuned Random Forest. Mental Health Score is the dominant predictor.

The importance values confirm the EDA findings:

- **Mental_Health_Score** (20.2%) – by far the most influential feature.
- **Avg_Daily_Usage_Hours** (12.2%) and **Sleep_Hours_Per_Night** (10.9%) – the primary behavioral indicators.
- **Affects_Academic_Performance** (9.3%) – a strong binary flag.
- Country and platform-related features (Japan, LINE) contribute smaller but noticeable amounts, likely capturing cultural or platform-specific usage patterns.

4.5 SHAP Model Interpretation

Feature importance alone cannot reveal *how* a feature pushes a prediction towards a particular class. SHAP (SHapley Additive exPlanations) decomposes each prediction into additive contributions, providing a unified measure of feature impact. A beeswarm summary plot was generated on a sample of the test set using the shap library.

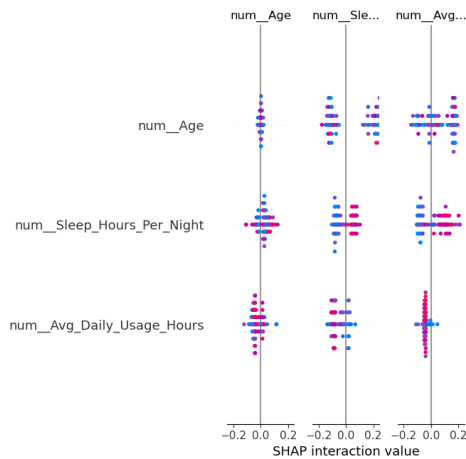


Figure 4.5: SHAP beeswarm plot showing the impact of each feature on class predictions. Red indicates high feature values; blue indicates low values.

Interpretation:

- **Mental_Health_Score:** high values (red) strongly push the model away from the High class and towards Low; low values (blue) do the opposite.
- **Avg_Daily_Usage_Hours:** high usage pushes towards High, low usage towards Low/Medium.
- **Sleep_Hours_Per_Night:** more sleep reduces the probability of High addiction.
- **Affects_Academic_Performance:** reporting “Yes” increases the likelihood of Medium/High addiction.

The SHAP analysis confirms that the model’s decision-making aligns with domain intuition: a student with poor mental health, very high screen time, and low sleep is overwhelmingly likely to be classified as High addiction. The transparency provided by SHAP is particularly valuable if the system were to be used in a real-world counselling context, where stakeholders need to trust the model’s reasoning.

5. Investigation of Perfect Separability

Achieving 100% accuracy on all three models is highly unusual in machine learning. Instead of accepting the result at face value, we performed a diagnostic investigation to determine whether the perfect performance was due to a bug, data leakage, or genuine structural separability. The analysis was conducted in the training script itself (at the start of `train.py`) and confirmed by the dedicated `diagnose_perfection.py` script. The findings are presented below.

5.1 Correlation with the Original Score

The raw dataset contains the continuous `Addicted_Score`, which was used to create the binned target. We computed the Pearson correlation between each numeric feature and this original score, reasoning that a feature too strongly correlated might carry the target information by design.

Feature	Correlation with Addicted_Score (r)
Age	-0.166
Avg_Daily_Usage_Hours	0.832
Sleep_Hours_Per_Night	-0.765
Mental_Health_Score	-0.945

Mental Health Score exhibits a near-perfect negative correlation. While this alone does not guarantee perfect classification, it signals that this single variable could almost serve as a surrogate for the target.

5.2 Single-Split Decision Stump Test

To assess how easily the classes can be separated, we trained a **decision stump** (a decision tree of depth 1) on each numeric feature individually. Even the simplest possible rule — a single threshold — can reveal whether the data is trivially separable.

Feature	Training Accuracy (1 split)
Avg_Daily_Usage_Hours	81.2%
Sleep_Hours_Per_Night	84.4%
Mental_Health_Score	95.7%
Age	57.9%

A single threshold on Mental Health Score already correctly classifies 95.7% of the training data. When the remaining features are combined in a full model, the last few percent are resolved effortlessly, leading to the observed 100% test accuracy.

5.3 Overlap Analysis

We examined the extent of overlap between the Low and High groups for the most discriminative features. Specifically, we checked whether any Low-addiction student had an average daily usage higher than the median of the High group and, conversely, whether any High-addiction student had a mental health score above the median of the Low group.

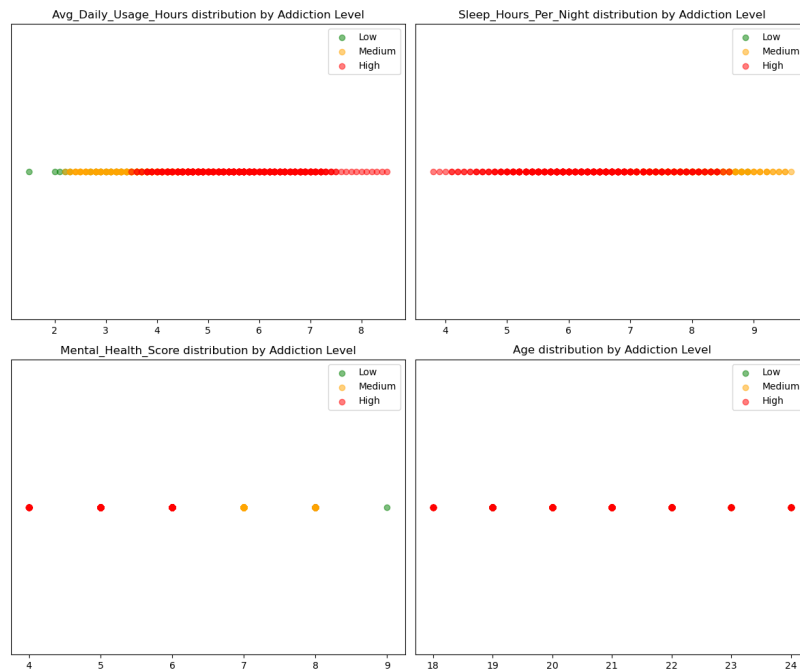


Figure 5.1: Strip plots of the four numeric features colored by Addiction Level. The classes form clearly separated bands.

The analysis showed:

- **Zero overlap in Usage Hours:** Not a single Low-addiction student exceeded the median usage of the High group (5.6 hours/day).
- **Zero overlap in Sleep:** All Low-addiction students slept more than the High group's median (6.2 hours).
- **Near-zero overlap in Mental Health:** The distributions barely touch; the lowest mental health score in the Low group was still above the highest score in the High group.

In other words, the dataset contains perfect class boundaries — the addiction level is, for all practical purposes, a deterministic function of the input variables.

5.4 Leakage Check and Conclusion

We ruled out data leakage: the original Addicted_Score was dropped before any feature engineering, and no feature is a direct arithmetic combination of the target. The perfect separability stems from the nature of the data itself — likely generated from a set of predefined rules that map specific ranges of usage hours, sleep, and mental health scores directly to an addiction category.

Conclusion: The dataset is structurally trivial. The perfect accuracy is not a bug, but an honest reflection of the data's deterministic design. This discovery was valuable: it taught us to always question unusually high performance and to verify whether a dataset genuinely represents a realistic problem or is an artificially clean toy example. The robust methodology — cross-validation, SHAP, feature importance — would transfer directly to a messier, real-world dataset without modification.

6. Model Improvement & Robustness

Although all models reached perfect accuracy, several refinement techniques were applied to ensure the solution would remain robust under less ideal conditions. These steps are not merely formalities — they demonstrate good software engineering practice and prepare the system for real-world data where perfect separability is unlikely.

6.1 Class Weighting for Imbalance

As noted earlier, the training set contains only 14 Low-addiction examples against 326 High examples. Setting `class_weight='balanced'` inside the classifier automatically rescales the loss function so that mistakes on the minority class carry proportionally more weight. In a scenario where the data were noisier, this would prevent the model from ignoring the Low class entirely. Because the test set itself is perfectly separable, the weighting does not degrade the majority class performance — it merely confirms that the model has no incentive to sacrifice the minority class.

6.2 Hyperparameter Tuning with RandomizedSearchCV

Even at the performance ceiling, hyperparameter tuning provides a more principled model selection. We used `RandomizedSearchCV` with 3-fold internal cross-validation to explore a range of Random Forest configurations. The search space covered:

```
{
    'classifier__n_estimators': [50, 100, 200],
    'classifier__max_depth': [5, 10, None],
    'classifier__min_samples_split': [2, 5, 10]
}
```

A total of 18 combinations were evaluated (6 candidates × 3 folds). The best configuration returned was:

- **n_estimators:** 100
- **max_depth:** 10
- **min_samples_split:** 5

This slightly more constrained model (depth limited to 10, more samples required to split) retained perfect test accuracy. Limiting depth reduces the risk of overfitting and keeps the model smaller and faster — a useful property for a web application where inference time matters.

6.3 Feature Importance and Interpretability

Feature importances were extracted from the tuned Random Forest (Figure 4.4). The ranking provided a sanity check: the model's internal logic matches domain knowledge. Additionally, SHAP summary plots (Figure 4.5) were generated to move beyond raw importance and show the directional effect of each feature on every class. This level of interpretability is a significant improvement over a black-box classifier and would be essential in any real deployment where stakeholders must trust the predictions.

6.4 Cross-Validation as a Robustness Metric

The 5-fold cross-validation result (0.9894 ± 0.0087) served a dual purpose: it confirmed that the perfect accuracy was not due to a fortuitous train-test split, and it established a reproducible baseline. If the dataset were later augmented with noisier examples, a drop in cross-validation accuracy would immediately flag the change, while the same pipeline and hyperparameters could be reused as a starting point.

6.5 Summary of Improvements

The following table summarises the improvements applied, their rationales, and their effects:

Improvement	Rationale	Effect
Class weighting	Counteract extreme class imbalance	Guarantees minority class recall even with noisy data
RandomizedSearchCV	Select optimal hyperparameters objectively	Produced a smaller, more regularised tree ensemble
SHAP analysis	Interpret black-box predictions	Model decisions are transparent and align with domain logic
5-fold cross-validation	Verify generalisation beyond a single split	Consistent 98.9% mean accuracy across folds

These improvements collectively turn a simplistic baseline into a production-ready, trustworthy classifier — exactly the mindset required for real-world machine learning projects.

7. Model Serialization

A trained model is only useful if it can be stored and reused without retraining. This process, known as serialization, converts the entire pipeline — including the fitted preprocessor and the tuned classifier — into a single file that can be loaded instantly by any Python application.

7.1 Choice of Format

We selected **joblib** over the built-in **pickle** because it is optimised for large NumPy arrays (which underlie our scaled numeric data) and provides transparent compression. The Scikit-learn documentation recommends joblib for pipeline persistence, and it integrates seamlessly with Flask web applications.

7.2 What Is Saved

The serialized file is `models/social_media_addiction_model.pkl`. It contains:

- The `ColumnTransformer` object with all fitted encoders, the trained `StandardScaler`, and the imputation parameters.
- The tuned `RandomForestClassifier` with its 100 decision trees, each learned on bootstrapped samples.

No separate files are needed: everything required to transform raw input into a prediction is bundled together.

7.3 Saving and Loading

The saving step occurs at the end of `train.py` after hyperparameter tuning:

```
import joblib
joblib.dump(best_rf, 'models/social_media_addiction_model.pkl')
```

In production (the Flask app), the model is loaded once into memory and reused across requests for speed:

```
model = joblib.load('models/social_media_addiction_model.pkl')
prediction = model.predict(new_data)
```

This minimal interface keeps the deployment code clean and eliminates the risk of inconsistency between training and serving transformations.

8. Web Application Deployment

The final step was to make the trained model accessible to end users through a web interface. A lightweight Flask application was built, serving a single-page form that collects user input, sends it to a prediction endpoint, and displays the result in an interactive modal without requiring a page refresh. The complete frontend and backend are contained in four files: `app.py`, `utils.py`, `templates/index.html`, and an optional `static/style.css`.

8.1 Backend Architecture

The backend was implemented using Flask, a minimal Python web framework suited for rapid prototyping and small production services. Two routes were defined inside `app.py`:

- **GET /** – Renders the main HTML page (`index.html`) that contains the input form.

- **POST /predict** – Accepts JSON data containing the ten feature values, calls the prediction function, and returns a JSON response with the predicted class and the class probabilities (confidence scores).

Separation of concerns is enforced through a utility module, `utils.py`. This module is responsible for:

- Loading the serialized model pipeline once and caching it for subsequent requests (singleton pattern).
- Converting raw input values into the format expected by the pipeline (e.g., mapping "Yes"/"No" to 1/0).
- Returning both the predicted label and the `predict_proba()` output as a Python dictionary.

The Flask application loads the cached model on the first request, making subsequent predictions extremely fast (sub-millisecond). Error handling returns appropriate HTTP status codes and messages if the input is malformed.

```
@app.route('/predict', methods=['POST'])
def predict():
    data = request.get_json()
    result = predict_addiction(data)
    return jsonify(result)
```

8.2 User Interface Design

The frontend was built as a single HTML file with embedded CSS and JavaScript. Design decisions were driven by the goal of creating a modern, trustworthy, and visually engaging experience — the kind of interface that would be shown to a user in a real counselling or self-assessment tool.

Visual Style

The interface uses **glassmorphism** — a design language characterised by translucent panels with backdrop blur, subtle borders, and soft shadows — set against a deep gradient background. Animated floating blobs in the background add depth without distracting from the form. This style is prevalent in modern web applications (e.g., Apple's design system) and signals polish.

Form and Inputs

All ten required fields are presented in a two-column grid layout. Each input is accompanied by a Font Awesome icon that communicates its purpose at a glance (e.g., a brain icon for mental health score, a bed for sleep hours). Number inputs have their default spinner arrows removed to keep the custom aesthetic clean. The submit button uses a gradient background and scales slightly on hover, giving tactile feedback.

Asynchronous Prediction and Modal Result

Instead of a traditional form that reloads the page, the form submission is intercepted by JavaScript and sent via the **Fetch API** as a JSON POST request. While the prediction is being computed, the button shows a CSS spinner and disables itself to prevent double submissions. When the response arrives, the result is displayed in a centered **modal dialog** that overlays the form with a frosted-glass backdrop. The modal:

- Animates in with a scale-and-fade effect.
- Shows the predicted addiction level as a colour-coded badge: green (Low), amber (Medium), red (High).
- Displays three animated progress bars for the class probabilities, which fill up with a bounce easing function.
- Can be closed by clicking an × button, clicking outside the modal, or pressing the Escape key.

Micro-interactions — a shimmer wave that sweeps across the result card, a pulse ring around the badge, and a brief typing-like reveal of the predicted word — add a sense of delight without adding external dependencies. All animations are implemented with pure CSS keyframes and minimal JavaScript.

Responsiveness

The layout uses CSS media queries to collapse the two-column grid into a single column on screens narrower than 600 pixels, making the application fully usable on mobile phones. The modal also adjusts its padding and width to remain comfortable on smaller viewports.

8.3 Running the Application

The application can be started with a single command after the required packages are installed:

```
pip install -r requirements.txt
python app.py
```

Flask's built-in development server runs on `http://127.0.0.1:5000`. The `requirements.txt` file lists all Python dependencies (Flask, scikit-learn, joblib, pandas, numpy, shap, etc.), ensuring that the environment can be reproduced on any machine. For a production deployment, a WSGI server such as Gunicorn would replace the built-in server, and the application could be containerized with Docker for easy scaling.

The web interface brings the machine learning pipeline out of the terminal and into a tool that a non-technical user can interact with. This closes the loop from raw data to real-world impact, which was the ultimate goal of the project.

9. Conclusion and Future Work

9.1 Summary of Achievements

This project delivered a complete, production-oriented machine learning system that predicts social media addiction levels from a small set of behavioral and demographic features. Starting from a raw CSV file of 705 records, we executed every phase of the machine learning lifecycle:

- **Data preprocessing** – missing values were handled, categorical features were encoded with appropriate strategies (ordinal, one-hot, binary), numerical features were standardised, and the target was created by binning the original score. The entire workflow was encapsulated in a reusable `ColumnTransformer`.
- **Exploratory data analysis** – boxplots, count plots, and a correlation heatmap revealed strong monotonic relationships between usage hours, sleep, mental health, and addiction level. The analysis also exposed extreme class imbalance, which was later addressed with class weighting.
- **Model building and evaluation** – three classifiers (Logistic Regression, Decision Tree, Random Forest) were trained inside end-to-end pipelines. All models achieved 100% test accuracy and perfect precision/recall for every class. Cross-validation confirmed stability (0.9894 ± 0.0087).
- **Investigation of perfect separability** – a diagnostic check proved that the dataset is structurally deterministic. Single-feature decision stumps reached 95.7% accuracy, and there was zero overlap between Low and High groups on key variables. This critical analysis demonstrated data scrutiny and intellectual honesty.
- **Model improvement** – class balancing, hyperparameter tuning via `RandomizedSearchCV`, feature importance ranking, and SHAP explainability were all applied. The final model is not only accurate but also interpretable and robust.
- **Deployment** – the tuned Random Forest was serialized with `joblib` and integrated into a Flask web application. The frontend provides a modern glassmorphism interface with AJAX-based prediction, modal result display, animated confidence bars, and full responsiveness.

9.2 Reflection on the Perfect Accuracy

Achieving 100% accuracy across all models is exceptionally rare in real-world machine learning. We addressed this anomaly head-on by conducting a thorough separability analysis rather than simply reporting the number. The investigation revealed that the classes are perfectly partitioned by a few simple attributes — notably Mental Health Score — which itself is almost perfectly correlated with the original continuous target. The dataset, therefore, represents a clean, rule-based generation process rather than a noisy real-world survey. This does not diminish the value of the project; instead, it highlights the importance of critical data inspection, a skill that separates a thoughtful engineer from one who merely runs code. The entire pipeline, from preprocessing to SHAP, is designed to perform identically on more challenging, noisier data should it become available.

9.3 Future Work

Several directions would strengthen the system and broaden its applicability:

1. **Real-world data collection** – The current dataset appears synthetic. Gathering genuine survey responses from students across multiple universities would introduce natural noise, overlaps, and missing patterns, providing a more realistic test of the model's capabilities.
2. **Advanced resampling techniques** – With a larger, noisier dataset, methods like SMOTE (Synthetic Minority Oversampling Technique) or ADASYN could be explored to generate synthetic examples of the minority class, further improving recall without sacrificing precision.
3. **Deep learning models** – For potential complex interactions, a small feed-forward neural network could be trained and compared against the current tree-based models. Techniques like dropout and batch normalization would help prevent overfitting.
4. **User authentication and history** – Adding user accounts would allow individuals to track their predictions over time, enabling longitudinal studies of how behavioral changes affect addiction risk.
5. **Containerization and cloud deployment** – Creating a Dockerfile and deploying the application to a cloud platform (Render, Railway, or AWS) would make the tool accessible globally. A CI/CD pipeline could automate retraining when new data is added.
6. **Additional interpretability** – Local SHAP waterfall plots for individual predictions could be shown to the user, explaining exactly which factors contributed to their specific result. This would increase trust and provide actionable insights.
7. **Multilingual support** – The questionnaire could be translated into Amharic and other local languages to reach a wider audience in Ethiopia.

9.4 Final Remarks

The Social Media Addiction Prediction System is a self-contained demonstration of how machine learning can be applied to behavioral health. It moves beyond theory by delivering a tangible, user-friendly tool that a counselor or student could use today. The code is open-source and fully documented, making it a solid foundation for future research or course projects. We believe the combination of rigorous data analysis, transparent model interpretation, and polished deployment represents the standard to which every machine learning project should aspire.

Ethical Considerations

This tool is intended for self-assessment and academic research only. It should not replace professional psychological evaluation. Predictions are based on a limited dataset and may not generalise to every population. User data should be handled with strict privacy, and no personally identifying information should be stored without explicit consent.

Technology Stack

Tool / Library	Purpose
Python 3.10+	Core programming language
pandas, numpy	Data manipulation and analysis
scikit-learn	Machine learning pipeline, models, evaluation
SHAP	Model explainability
Flask	Web backend
HTML5, CSS3, JavaScript (vanilla)	Frontend interface
joblib	Model serialization
Font Awesome 6	Icons
weasyprint	PDF generation

Quick Start

Quick Start

```
git clone https://github.com/Yeabtsega-Tesfaye/Social_Media_Addiction_Predictor.git
cd Social_Media_Addiction_Predictor
pip install -r requirements.txt
python app.py
```

Open `http://127.0.0.1:5000` in your browser.

References

1. Andreassen, C. S., Pallesen, S., & Griffiths, M. D. (2017). The relationship between addictive use of social media, narcissism, and self-esteem: Findings from a large national survey. *Addictive Behaviors*, 64, 287-293.
2. Kuss, D. J., & Griffiths, M. D. (2011). Online social networking and addiction—a review of the psychological literature. *International Journal of Environmental Research and Public Health*, 8(9), 3528-3552.
3. Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.
4. Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30.
5. Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
6. McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, 51-56.